

Ficheros

Métodos de asignación

En los sistemas de ficheros, uno de los aspectos clave es la **gestión de espacio en disco duro** utilizado por los archivos. Existen distintos **métodos de asignación** de bloques que permiten almacenar y acceder a los datos de manera eficiente. A continuación, se presentan los métodos más comunes:

1. Contiguous Allocation

En este método, **cada archivo ocupa un conjunto de bloques contiguos** en el disco.

- **Ventajas:**
 - Ofrece el **mejor rendimiento** en la mayoría de los casos, ya que permite el acceso secuencial y directo eficiente.
 - Es un método **simple**: sólo se necesita conocer el número de bloque inicial y la longitud del archivo en bloques.
- **Desventajas:**
 - Requiere **conocer el tamaño de archivo de antemano**.
 - Dificultad para encontrar espacio contiguo para archivos grandes o crecientes.
 - Pueden necesitar **compartir** el disco, tanto **en línea** como **fuera de línea**.
 - Genera **fragmentación externa e interna**

2. Linked Allocation

Con la asignación enlazada, **cada archivo es una lista enlazada de bloques** dispersos por el disco.

- Cada bloque contiene **un puntero al siguiente** bloque del archivo.
- **Ventajas:**
 - No hay fragmentación externa.
 - Ideal para archivos de tamaño dinámico
- **Desventajas:**
 - No permite **acceso directo** ni aleatorio, ya que para llegar a un bloque específico es necesario recorrer los anteriores.
 - Mayor **sobrecarga** por el almacenamiento de punteros.
 - Puede requerir compactación.

3. File Allocation Table (FAT)

Es una variación del método enlazado que mejora el acceso:

- Una **tabla en el inicio del volumen** almacena los punteros de todos los bloques del disco.
- La tabla es **indexada por número de bloque**
- Se puede **cargar en memoria principal**, lo que mejora el rendimiento al permitir acceso directo a través de la tabla.
- **Ventajas:**
 - Evita la lectura secuencial de punteros en disco.
 - Asignar nuevos bloques es sencillo.

4. Index Allocation

Este método asigna a cada archivo uno o más **bloques índice**, que contienen **punteros directos** a los bloques de datos del archivo.

- Permite **acceso directo y dinámico** sin fragmentación externa.
- Cada archivo tiene su propia **tabla de índice**.
- **Ventajas:**
 - Soporta archivos de tamaño variable, ideal para operaciones que requieren acceso aleatorio.
- **Desventajas:**
 - Hay una **sobrecarga por los bloques de índice**.
 - Requiere mayor gestión para archivos grandes.

Sistema de ficheros UNIX

Introducción al sistema de archivos: visión lógica.

La estructura lógica típica de un sistema de archivos UNIX incluye:

- **BOOT:** contiene el gestor de arranque necesario para iniciar el sistema operativo.
- **SUPERBLOCK:** Mantiene información sobre el sistema de archivos, como tamaño, número de bloques, número de inodos, etc.
- **INODE LIST:** Estructura que almacena metadatos de los archivos.
- **DATA AREA:** Zona en la que se guarda el contenido real de los archivos.

Comandos relacionados

- `du` - Muestra el uso del disco por archivos y directorios.
- `df` - Muestra el uso del espacio en disco por sistemas de archivos.

- `mount / umount` - Monta y desmonta sistemas de archivos.
- `mkfs` - Crea un nuevo sistema de archivos.

Montaje de sistemas de archivos

El comando `mount` permite unir dos sistemas de archivos en uno solo. Por ejemplo, al insertar un dispositivo, este se puede montar dentro del sistema raíz con: `mount /dev/fd0 /mnt`

Ejemplos comunes:

- Montar un segundo disco: `mount -t ext4 /dev/hda1/ home2`
- Montar una unidad de CD: `mount -r -t iso9660 /dev/scd0 /cdrom`
- Montar una memoria USB: `mount -w -o noatime /dev/sda1 /memstick`
- Desmontar la USB: `umount /dev/sda1` o `umount /memstick`

Comandos útiles:

- `lsblk` - Muestra los dispositivos de bloques conectados.
- `blkid` - Muestra información sobre los dispositivos de bloques.
- `cfdisk`, `parted` - Herramientas de particionado.

Opción para mount	Descripción
auto / noauto	Montaje automático al inicio o solo bajo petición del usuario (<code>mount -a</code>).
user / nouser	Permite (o no) que usuarios sin privilegios monten el dispositivo.
exec / noexec	Permite (o no) ejecutar archivos binarios en el sistema de archivos.
ro	Monta el sistema en modo de solo lectura.
rw	Monta el sistema en modo lectura-escritura.
sync / async	Las operaciones de escritura se realizan de forma síncrona (seguras). Las escrituras se hacen de forma asíncrona (más rápidas pero arriesgadas).
default	Usa las opciones por defecto: <code>rw</code> , <code>suid</code> , <code>dev</code> , <code>exec</code> , <code>auto</code> , <code>nouser</code> , <code>async</code> .

Estructura típica de directorios en UNIX

Directorio	Descripción
/	Raíz del sistema de archivos.
/bin	Ejecutables básicos necesarios por todos los usuarios.

Directorio	Descripción
/sbin	Herramientas administrativas del sistema (<code>fsck</code> , <code>mount</code> , etc.).
/boot	Archivos necesarios para el arranque del sistema.
/dev	Representación en archivos de los dispositivos del sistema.
/etc	Archivos de configuración del sistema.
/home	Directorios personales de los usuarios.
/lib	Bibliotecas esenciales compartidas y módulos del kernel.
/media	Punto de montaje automático para dispositivos extraíbles.
/mnt	Punto de montaje manual para dispositivos o sistemas remotos.
/proc	Sistema de archivos virtual que representa procesos activos.
/root	Directorio personal del superusuario (root).
/tmp	Archivos temporales. Puede vaciarse al reiniciar.
/usr	Programas, bibliotecas y recursos no críticos del sistema.
/usr/bin	Ejecutables no esenciales (como <code>gcc</code> , <code>gedit</code> , etc.).
/usr/include	Cabeceras de desarrollo (<code>stdio.h</code> , <code>math.h</code> ...).
/usr/lib	Bibliotecas para programas en <code>/usr</code> y otros.
/var	Archivos variables como logs, colas de impresión y correo.
/var/log	Archivos de registro del sistema.
/var/email	Correos entrantes de los usuarios.
/var/spool	Colas de tareas (correo, impresión...).
/var/tmp	Archivos temporales que se conservan entre reinicios.

Bibliotecas estáticas y dinámicas

En UNIX, las bibliotecas de desarrollo están en **/usr/lib** y se clasifican en:

- **Bibliotecas estáticas:** *.a
- **Bibliotecas dinámicas:** *.so

Ejemplo en C:

```
#include <stdio.h>
#include <math.h>
int main()
{
    float x = 0.5, y;
```

```
y = sin(x);  
printf("sin(%f) = %f\n", x,y);  
return 0;  
}
```

- **Enlace estático:** `gcc ejemplo.c -l,`
Se enlaza con `/usr/lib/libm.a`, incluyendo el código de `sin()` directamente en el ejecutable.
Resultado: binario más pesado.
- **Enlace dinámico:** `gcc ejemplo.c -lm`
Usa `/usr/lib/libm.so`. El código de `sin()` se carga en tiempo de ejecución. Ventaja: menos espacio, y las bibliotecas pueden actualizarse sin recompilar.

Visión física de las particiones del disco

Particiones del disco en un PC

En los sistemas UNIX y similares, el disco duro se divide físicamente en **particiones**, que permiten organizar el almacenamiento en secciones independientes. Cada partición puede contener un sistema de archivos diferente, y una de ellas puede estar dedicada al arranque del sistema.

MBR (Master Boot Record) por dispositivo de almacenamiento: El **MBR**, o Registro de Arranque Maestro, es una pequeña sección al principio de un dispositivo de almacenamiento, cuya función es contener el cargador de arranque y la tabla de particiones principal. El MBR ocupa los **primeros 512 bytes** del disco y está presente en **cada dispositivo de almacenamiento**, no por partición.

Sector de arranque por partición: Cada partición que contenga un sistema operativo debe tener su **sector de arranque**. Este sector incluye código específico, como el cargador de arranque del sistema, que se encarga de iniciar la carga del sistema operativo.

Registro de arranque de partición extendida: Cuando el número de particiones primarias no es suficiente, se puede crear una **partición extendida**, dentro de la cual se pueden definir múltiples **particiones lógicas**. Cada partición lógica contiene su propio **registro de arranque de partición extendida**, permitiendo así gestionar más particiones dentro de un esquema limitado.

Estructura del Master Boot Record

El MBR está compuesto por:

1. **Código de arranque principal:** pequeño programa que localiza el sector de arranque de una partición activa y transfiere el control a él.
2. **Tabla de particiones:** contiene información sobre hasta cuatro particiones primarias.
3. **Firma de arranque:** valor fijo 0x55AA que indica un MBR válido.

Offset	Tamaño	Descripción
0x000	446 bytes	reservado
0x1BE	16 bytes	Entrada partición 1
0x1CE	16 bytes	Entrada partición 2
0x1DE	16 bytes	Entrada partición 3
0x1EE	16 bytes	Entrada partición 4
0x1FE	2 bytes	0xAA55

Limitaciones del esquema de particionado MBR

El esquema MBR, creado a principios de los años 80, presenta **limitaciones técnicas** importantes:

- Sólo permite **cuatro particiones primarias**.
- Máximo tamaño por partición: **2 TB**.
- No es compatible con discos modernos con tamaños superiores ni con algunos métodos avanzados de arranque como UEFI

GPT: La evolución del particionado

Para superar estas limitaciones, **Intel desarrolló en los años 90** un nuevo formato de tabla de particiones: el **GPT (GUID Partition Table)**. Actualmente, **la mayoría de los sistemas operativos modernos lo soportan** y es el esquema estándar en sistemas con firmware UEFI. Las ventajas de GPT incluyen:

- Permite un **número casi ilimitado de particiones**.
- Soporta discos de **más de 2TB**.
- Mayor fiabilidad y redundancia gracias a la presencia de **copias de respaldo** de la tabla de particiones.

System V vs BSD

Los sistemas UNIX han utilizado diferentes estructuras lógicas de sistema de archivos a lo largo del tiempo. Dos de las más influyentes son el **System V File System** y el **BSD Fast File System**. A continuación se detalla su organización lógica y las diferencias clave entre ambos.

Estructura lógica en System V

El sistema de archivos de **System V** organiza el disco en cuatro zonas principales.

1. **BOOT**: contiene el código de arranque necesario para iniciar el sistema operativo desde esa partición.
2. **SUPERBLOQUE**: almacena información general sobre el sistema de archivos, como el tamaño total, el número de inodos, bloques libres, etc.
3. **Lista de inodos**: colección de estructuras que describen los archivos.
4. **Área de datos**: espacio del disco donde se almacena el contenido real de los archivos y directorios.

Estructura lógica en BSD

El **BSD Fast File System** mejora la organización del disco dividiéndolo en **grupos de cilindros**, lo cual favorece el rendimiento y reduce la fragmentación. Cada grupo de cilindros contiene sus propias copias de estructuras clave, lo que mejora la fiabilidad y el acceso local a los datos.

La organización es la siguiente:

1. **Boot**: código de arranque.
2. **Superbloque**: bloque principal con la información del sistema de archivos.
3. **Grupos de cilindros**:
 - CG 0
 - CG 1
 - ...
 - CG N

Cada grupo de cilindros contiene:

- **superbloque (replicado)**: copia del superbloque para mejorar la tolerancia a fallos.
- **Cabecera del grupo de cilindros**: información de control sobre el grupo.
- **Lista de inodos del grupo i**: describe los archivos asignados a ese grupo.
- **Área de datos del grupo i**: donde se almacenan los datos reales de los archivos.

Organización del disco en grupos de cilindros.

La división en grupos de cilindros permite:

- **Adaptar inodos y datos cercanos físicamente** para mejorar el rendimiento.
- Reducir el **tiempo de búsqueda** del cabezal del disco.
- Tener **copias de seguridad parciales** de estructuras críticas, como el superbloque.

Bloques y fragmentos en BSD

El sistema BSD utiliza una estrategia eficiente para el uso del espacio en disco mediante **bloques grandes** y **fragmentos**. Esta técnica permite reducir el desperdicio de espacio, especialmente en archivos pequeños.

Funcionamiento:

- Los archivos se almacenan en bloques grandes.
- El **último bloque** de archivo, si no está completamente lleno, se almacena como un **fragmento** más pequeño.

Esta técnica ayuda a **minimizar la fragmentación interna** y aprovechar mejor el espacio disponible en disco.

Inodos

El sistema operativo asocia un **inodo** a cada archivo.

Esta estructura contiene toda la información necesaria para gestionar el archivo, salvo su nombre, que se almacena en el directorio.

Es importante **diferenciar** entre:

- **Inodos en disco:** ubicados en la *Lista de Inodos*, forman parte permanente del sistema de archivos.
- **Inodos en memoria:** almacenados en la *Tabla de Inodos*, se gestionan dinámicamente y su estructura es similar a la de la *Buffer cache*.

Estructura de los Inodos

Cada inodo en disco contiene la siguiente información sobre el archivo:

- **Propietario:** identificador del usuario dueño del archivo.
- **Grupo:** grupo de usuarios asociado al archivo.
- **Tipo de archivo:** indica si es un archivo regular, directorio, enlace simbólico, etc.
- **Permisos de acceso:** lectura, escritura y ejecución para propietario, grupo y otros.
- **Fechas del archivo**
 - Último acceso.
 - Última modificación del contenido
 - Última modificación del propio inodo
- **Número de enlaces:** número de hard links que apuntan a este inodo.
- **Tamaño del archivo:** en bytes
- **Direcciones de disco:** punteros a los bloques del disco donde se almacenan los datos reales del archivo

Tipos de archivo y permisos de archivo

Los **tipos de archivo** y los **permisos de acceso** son gestionados mediante el *modo del archivo*, una parte del inodo.

Comandos relacionados:

- `chmod` : cambia los permisos de acceso de un archivo o directorio. También existe la `syscall` `chmod()` .
- `chown` : cambia el propietario de un archivo o directorio. Existe también como `syscall` `chown()` .

Cambiar los permisos con chmod

Hay **dos formas de sintaxis**:

1. **Modo simbólico:** `chmod u+rw g-x o+x archivo.txt`
 - u = usuario, g = grupo, o = otros
 - + añade permisos, - los elimina
2. **Modo simplificado para todos los usuarios:** `chmod a+rx archivo.txt`
 - a = todos
 - r = lectura, w = escritura, x = ejecución.

"Sticky bit"

Significado tradicional en archivos ejecutables:

- Indicaba que el **código del programa** se mantenía en memoria tras su ejecución para acelerar futuros usos.
- Actualmente, **el kernel de Linux ignora el sticky bit en archivos**.

Significado actual en directorios:

Se usa para **proteger archivos en directorios públicos** como `/tmp`.

Es un directorio con sticky bit: **Sólo el superusuario, el dueño del directorio o el dueño del archivo** pueden borrar o renombrar entradas dentro del directorio.

Fechas de los archivos

Cada archivo en UNIX almacena **tres marcas de tiempo**, accesibles mediante la estructura `stat`:

- `st_atime` : Fecha del último acceso.
- `st_mtime` : Fecha de la última modificación de datos.

- **st_ctime**: Fecha de última modificación del inodo.

Links

Conceptos clave:

- **Hard links:**
 - Dos o más entradas de directorio que **apuntan al mismo inodo**.
 - Son **idénticos al archivo original**: comparten contenido, permisos y metadatos del inodo.
 - **No se pueden crear hard links a directorios ni entre sistemas de archivos diferentes**
- **Symbolic link:**
 - Es un **archivo independiente** que contiene la **ruta** hacia otro archivo o directorio.
 - Puede apuntar a **archivos o directorios**, incluso si están en otro sistema de archivos o partición.

Estructura del diseño por bloques en el disco

Cada archivo en el sistema de archivos UNIX está asociado a:

- Un **inodo** en la **lista de inodos**-
- Una serie de **bloques de datos** en el área de datos del disco.

Bloques de datos del archivo

El inodo contiene **direcciones de disco** que permiten acceder a los bloques que contienen el contenido del archivo. El acceso a estos bloques se organiza jerárquicamente para permitir el crecimiento eficiente del archivo, aunque **el acceso se vuelve más lento a medida que el archivo crece**, debido a los niveles de indirección.

Esquema de direcciones de bloque

Suponiendo:

- Tamaño del bloque **1 Kbyte (1024 bytes)**
- Tamaño de una dirección: **4 bytes**

Un inodo típico incluye:

- **10 direcciones directas** -> Acceso directo a bloques de datos.
- **1 dirección de bloque indirecto simple** -> Apunta a un bloque que contiene 256 direcciones directas a bloques de datos.

- **1 dirección de bloque indirecto doble** -> Apunta a un bloque que contiene 256 direcciones a otros bloques, que a su vez contienen 256 direcciones a bloques de datos.
- **1 dirección de bloque indirecto triple** -> Apunta a un bloque que contiene 256 direcciones a bloques, que contienen 256 direcciones a otros bloques, que finalmente contienen 256 direcciones a bloques de datos.

Cálculo del tamaño máximo de un archivo

1. Direcciones directas (10 bloques)

Cada dirección apunta directamente a un bloque de datos.

- **Fórmula:** $10 \times 1 \text{ Kbyte}$
- **Capacidad:** 10 Kbytes

2. Bloque indirecto simple

Un bloque de punteros que almacena $1024/4 = 256$ direcciones.

- **Fórmula:** $10 \text{ Kbytes} + (256 \times 1 \text{ Kbyte})$
- **Capacidad total:** 266 Kbytes

3. Bloque indirecto doble

Apunta a 256 bloques indirectos simples.

- **Fórmula:** $10 \text{ Kbytes} + 256 \text{ Kbytes} + (256^2 \times 1 \text{ Kbyte})$
- **Capacidad total:** $\approx 64.26 \text{ Mbytes}$

4. Bloque indirecto triple

Apunta a 256 bloques indirectos dobles.

- **Fórmula:** $10 \text{ Kbytes} + 256 \text{ Kbytes} + 64 \text{ Mbytes} + (256^3 \times 1 \text{ Kbyte})$
- **Capacidad total:** $\approx 16.32 \text{ Gbytes}$

Directorios

Un **directorio** es un archivo cuyo contenido se interpreta como una **lista de entradas de directorio**. Estas entradas asocian un **nombre de archivo** con el **número de inodo** que lo representa en el sistema de archivos.

Formato de las entradas de directorio.

Los formatos pueden variar según la implementación del sistema operativo:

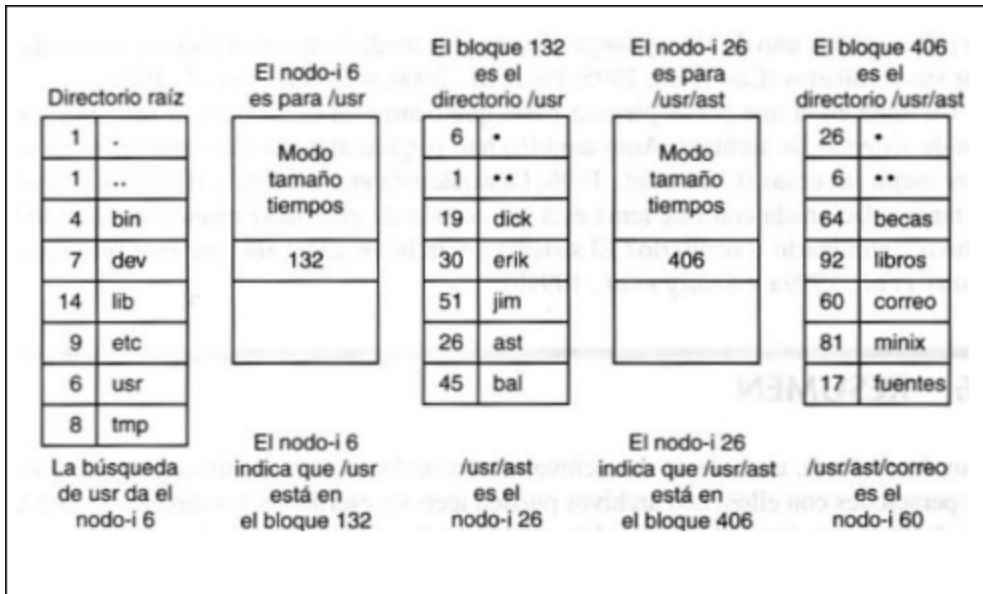
- **Formato System V:**
 - **Número de inodo:** 2 bytes
 - **Nombre del archivo:** 14 bytes
- **Formato BSD:**
 - **Número de inodo:** 4 bytes
 - **Longitud de la entrada:** 2 bytes

- **Longitud del nombre del archivo:** 2 bytes
- **Nombre del archivo:** cadena terminada en \0, completada hasta múltiplo de 4 bytes

Las funciones estándar para manipular directorios están definidas en **<dirent.h>**:

- **opendir()** - Abre un directorio para su lectura.
- **readdir()** - Lee una entrada del directorio
- **closedir()** - Cierra el descriptor del directorio.

Cálculo del número mínimo de accesos al disco



En el caso **óptimo**:

- **Bloques leídos del área de datos:**
 - Bloque del directorio /
 - Bloque 132 (de /usr)
 - Bloque 406 (de /usr/ast)

Total **3 accesos**.
- **Accesos a inodos en la lista de inodos del disco:**
 - Se acceden a los inodos: 1, 6, 26 y 60

Total: **4 inodos**

Pero, cuántos bloques de disco se leyeron realmente?

- Si asumimos que los 4 inodos están almacenados en un **mismo bloque de inodos**, entonces:
 - Sólo se necesita **una lectura de bloque** de la lista de inodos.

Estructuras del kernel: sistema de archivos

El kernel del sistema operativo utiliza diversas estructuras para gestionar el acceso a los archivos y optimizar el rendimiento del sistema.

Una de las más importantes es el **Buffer Caché**, que regula el flujo de datos entre los dispositivos de almacenamiento secundario y el núcleo. Su función principal es **reducir el número de accesos físicos al disco**, almacenando temporalmente bloques de datos en memoria. De forma similar, existe un mecanismo asociado a la **memoria virtual**, conocido como **Page Cache**, que actúa como una caché para páginas de memoria, incluyendo aquellas relacionadas con archivos mapeados en memoria o accedidos frecuentemente.

Esquema de principales estructuras del kernel del sistema de archivos

Las estructuras clave del kernel en la gestión del sistema de archivos incluyen:

- **Superbloque**: Contiene información general del sistema de archivos, como su tamaño, número de inodos, y localización de bloques de datos.
- **Lista de inodos en memoria**: Conjunto de inodos cargados desde disco a memoria. Cada inodo representa un archivo o directorio, con metadatos como permisos, tamaño y punteros a bloques de datos.
- **Tabla de archivos abiertos por el sistema**: Contiene una entrada por cada archivo abierto en el sistema, con punteros a su inodo y posición actual de lectura / escritura.
- **Tabla de descriptores de archivo por proceso**: Cada proceso mantiene su propia tabla de descriptores que apuntan a las entradas de la tabla global de archivos abiertos.
- **Buffer Caché**: Área de memoria que almacena bloques de datos recientemente leídos o escritos, para evitar accesos repetidos al disco.
- **Page Cache**: Memoria caché de páginas virtuales, muchas de las cuales corresponden a datos de archivos usados por los procesos.

Llamadas al sistema para el sistema de archivos

El sistema operativo proporciona un conjunto de **llamadas al sistema** para permitir que los procesos puedan interactuar con los archivos. Estas funciones están definidas en bibliotecas del sistema como `<fcntl.h>` y `<unistd.h>`, y permiten operaciones como abrir, leer, escribir o duplicar descriptores de archivo.

```
int open(char *nombre, int modo, int permisos);
```

Abre un archivo y devuelve un **descriptores de archivo**, que será un número entero que representa el archivo abierto.

- Nombre: ruta del archivo
- Modo: modo de apertura

- Permisos: permisos (utilizados con O_CREAT para indicar los permisos iniciales del archivo).

Modos posibles

Pueden combinarse con operadores OR |

- 0: sólo lectura
- 1: sólo escritura
- 2: lectura y escritura

O usando constantes de `<fcntl.h>`

- `O_RDONLY`: sólo lectura.
- `O_WRONLY`: sólo escritura.
- `O_RDWR`: lectura y escritura.
- `O_APPEND`: añadir al final.
- `O_CREAT`: crear archivo si no existe.

```
int read(int df, char *buff, int n);
```

Lee datos del archivo referenciado por el descriptor `df` y los almacena en el búfer `buff`.

- `df`: descriptor del archivo abierto.
- `buff`: dirección en el espacio del usuario donde se guardan los datos.
- `n`: número de bytes a leer

```
int write(int df, char *buff, int n);
```

Escribe en el archivo referenciado por `df` los datos contenidos en el búfer `buff`.

- `df`: descriptor del archivo abierto.
- `buff`: dirección de los datos a escribir.
- `n`: número de bytes a escribir.

Duplicación de descriptors

```
int dup(int df);
```

Crea un **nuevo descriptor de archivo** que referencia al **mismo archivo abierto** que el descriptor `df`. Devuelve el **primer descriptor libre disponible**.

```
int dup2(int df, int newfd);
```

Hace que `newfd` sea una copia exacta de `df`. Si `newfd` ya estaba en uso, se cierra automáticamente antes de ser reutilizado.

Ejecutables con SETUID

El kernel de UNIX asocia **dos identificadores de usuario** a cada proceso:

1. **UID real**: identifica al usuario que ha lanzado el proceso.
2. **UID efectivo**: se utiliza para verificar permisos de acceso a archivos, asignar la propiedad de los archivos recién creados y controlar el envío de señales a otros procesos.

El kernel permite que un proceso cambie su UID efectivo al ejecutar un programa con **bit SETUID activado**, o al invocar explícitamente la llamada al sistema `setuid()`

¿Qué es un programa SETUID?

Es un ejecutable que tiene activado el bit SETUID en su campo de permisos. Cuando se ejecuta, el kernel **establece al UID efectivo como el propietario del archivo ejecutable**, en lugar del UID real del proceso.

Notas sobre cifrado y autenticación

Además del clásico **DES**, hoy en día se utilizan algoritmos más avanzados como **AES (Advanced Encryption Standard)** con tamaños de clave de 128, 192 o 256 bits.

La mayoría de sistemas Linux emplean **funciones hash** para la autenticación:

- MD5: produce un hash de 128 bits.
- SHA-1: produce un hash de 160 bits.

Llamada al sistema `setuid()`

```
int setuid(uid_t uid);
```

Esta llamada cambia el UID efectivo del proceso y su comportamiento depende del UID efectivo actual.

Casos en los que tiene éxito

1. **Si el proceso es ejecutado por el superusuario**

El kernel cambia tanto el UID real como el efectivo al valor pasado como parámetro.

2. **Si el proceso *no* es ejecuta por el superusuario:**

- Si uid coincide con el UID real del proceso -> éxito
- Si uid coincide con el UID efectivo previamente guardado -> éxito.
- En cualquier otro caso -> error.

Sistema de archivos Linux Ext2fs

El sistema de archivos **Ext2fs** (Second Extended File System) utiliza una estrategia similar a la del sistema de archivos BSD Fast File System (**FFS**) para localizar los bloques de datos asociados a un archivo determinado.

Pero, existen diferencias significativas entre ambos, especialmente en sus políticas de asignación de disco.

Comparación con FFS:

- En **FFS**, los archivos se almacenan en bloques de 8 KB, los cuales pueden dividirse en **fragmentos de 1 KB**. Esto aprovecha mejor el espacio en disco al manejar archivos pequeños o bloques parcialmente llenos al final de un archivo.
- En contraste, **Ext2fs no utiliza fragmentos**. En su lugar, realiza la asignación directamente en bloques de tamaño menor.

Tamaños de bloque en Ext2fs:

- El tamaño por defecto de bloque en **Ext2fs** es de **1 KB**, aunque se admiten bloques de **2 KB** y **4 KB** dependiendo de la configuración al formatear el sistema de archivos.

Políticas de asignación de bloques:

- **Ext2fs** emplea políticas de asignación diseñadas para colocar bloques lógicamente contiguos de un archivo en bloques **físicamente contiguos en disco**.
- Esta estrategia mejora el rendimiento, ya que permite al sistema enviar **una única petición de E/S** para leer o escribir múltiples bloques, reduciendo así el número de accesos al disco.

Sistema de archivos con Journaling

Los sistemas operativos mantienen un *Buffer caché* de los datos y metadatos de los archivos para optimizar el acceso al sistema de archivos.

Problema: Cuando ocurre un **apagón eléctrico** o un **fallo del sistema**, puede haber **inconsistencias** en el sistema de archivos si los datos modificados en la caché aún no se han escrito en disco.

Comando relacionado: **fsck** (file system check). Herramienta que verifica y repara inconsistencias en sistemas de archivos no transaccionales.

Journaling

Un **sistema de archivos con journaling** es un sistema **resiliente a fallos**, que garantiza la integridad de los datos. La clave de su funcionamiento es:

- Antes de actualizar los bloques originales en disco, los **metadatos** se escriben primero en un **registro secuencial**.
- Sólo cuando esta operación se ha completado, se procede a escribir los datos en disco.
- En caso de fallo, el código de recuperación analiza el journal y **reproduce las operaciones pendientes**, resultando así la coherencia del sistema sin necesidad de verificar todo el

disco.

Sistema de archivos con journaling en Linux:

- ext3
- ext4
- ReiserFS
- XFS
- JFS

Ventaja de journaling

Un sistema con journaling no necesita revisar todo el sistema de archivos. Sólo los **inodos y directorios** implicados en la operación fallida, mejorando así la velocidad y eficiencia de la recuperación.

1. [Introducción a sistemas operativos](#)
2. [Ficheros](#)
3. [Memoria](#)
4. [Procesos](#)
5. [Entrada Salida](#)